

UNIVERSIDAD TECNOLÓGICA DE PEREIRA
PROGRAMA DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

Informe Técnico: Sistema de Juego de Bingo

Práctica 2 - Programación Orientada a Objetos

Integrantes:

José Angel Mejía Medina
Henry Román Puerres Tipas
Santiago Ordóñez Ordóñez
Jordan Valencia Patiño

Clase:

Programación IV G102

24 de abril de 2026

1 Estructura de las Clases Implementadas

A continuación se detallan los atributos y métodos de las entidades principales del sistema desarrollado en Python.

1.1. Clase: Juego

Es la clase directora que coordina la partida y la interacción entre los participantes y el bombo.

■ **Atributos:**

- `_bombo` (Bombo): Instancia del extractor de números.
- `_jugadores` (List[Jugador]): Lista de jugadores registrados.
- `_ganador` (Optional[Jugador]): Referencia al jugador que canta bingo.

■ **Métodos:** `registrar_jugador()`, `ejecutar_turno()`, `reporte_final()`.

1.2. Clase: Bombo

Gestiona el conjunto de números y garantiza una extracción aleatoria sin repetición.

■ **Atributos:**

- `_disponibles` (List[int]): Números que aún no han salido.
- `_historial` (List[int]): Registro de números extraídos.

■ **Métodos:** `extraer()`, `hay_numbers()`.

1.3. Clase: Jugador

Representa al usuario que participa en la partida con uno o más cartones.

■ **Atributos:**

- `nombre` (str): Identificador del usuario.
- `_cartones` (List[Carton]): Colección de cartones asignados.
- `_numeros_marcados` (int): Contador de aciertos en la partida.

■ **Métodos:** `agregar_carton()`, `notificar_numero()`.

1.4. Clase: CartonDoble (Subclase de Carton)

Especialización que añade la funcionalidad de manejar dos grillas de juego independientes.

■ **Atributos:**

- `segunda_tarjeta`: Grilla adicional generada automáticamente.
- `marcados_segunda`: Set de números marcados en la segunda grilla.

■ **Métodos:** `tiene_bingo()` (Overriding), `grilla_mas_cercana()`.

2 Mecanismos de Relación entre Clases

Para el diseño del sistema se implementaron los siguientes vínculos basados en los requerimientos de la práctica:

1. **Relación Juego - Bombo (Composición):** Se establece mediante la instanciación del Bombo dentro del método `__init__` de la clase `Juego`. El ciclo de vida del bombo está estrictamente ligado al del juego.
2. **Relación Juego - Jugador (Asociación):** El `Juego` utiliza un atributo de lista (`_jugadores`) para referenciar a los objetos `Jugador`. Estos se registran y retiran voluntariamente, por lo que su existencia es independiente del objeto `Juego`.
3. **Relación Jugador - Cartón (Agregación):** Los objetos `Carton` se pasan como parámetros al método `agregar_carton()`. El jugador conoce los cartones, pero si el jugador es eliminado, los cartones pueden ser reasignados a otro usuario.
4. **Relación Cartón - CartónDoble (Herencia):** Se utiliza la sintaxis `class CartonDoble(Carton)` permitiendo que la subclase herede los métodos de validación y generación, extendiendo su comportamiento para la verificación de doble bingo.